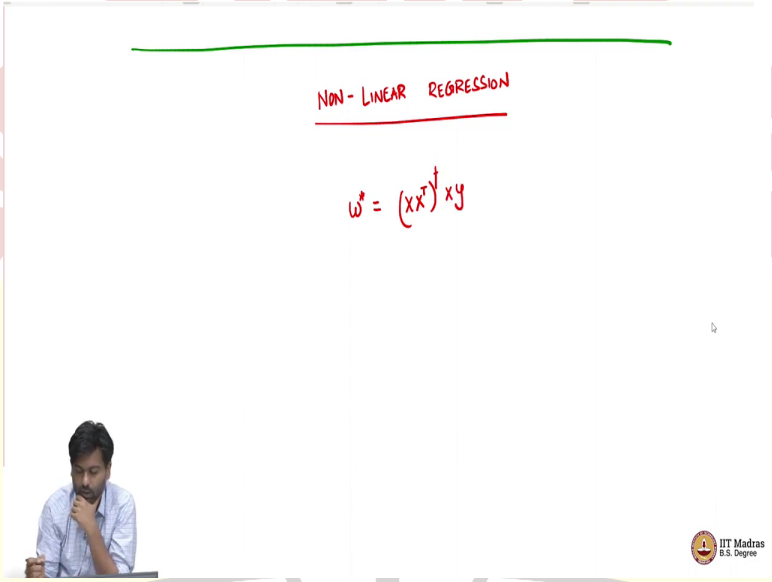
IIT Madras

ONLINE DEGREE

Machine Learning Techniques
Professor Arun Rajkumar
Department of Computer Science and Engineering
Indian Institute of Technology Madras
Kernel Regression

(Refer Slide Time: 00:13)



So, let us try to understand non-linear regression now. By non-linear, I mean that let us say we have some hypothesis that the actual the relation between features that should explain why it is not necessarily linear it is not $w^T x$. But then some other complicated relation, maybe height and weight are your features, you want height^2 + 3(weight) - weight^2, or something like that, that best explains our label y for example.

So, then your linear regression will fail, because it cannot capture these either the squared terms, or if you had a cross term like height into weight, which makes sense, which makes which is important to explain your y still linear regression would not be able to capture such relationships, you can still run linear regression, but then even the best w will make a lot of perhaps a lot of error in your training set and your validation set also.

So, how can we do non-linear regression? Remember, our idea for moving from linear PCA to non-linear PCA was via this notion of kernels. We will do the same thing here. The trick there was to write the optimization problem of PCA on what you wanted to get from PCA in terms of

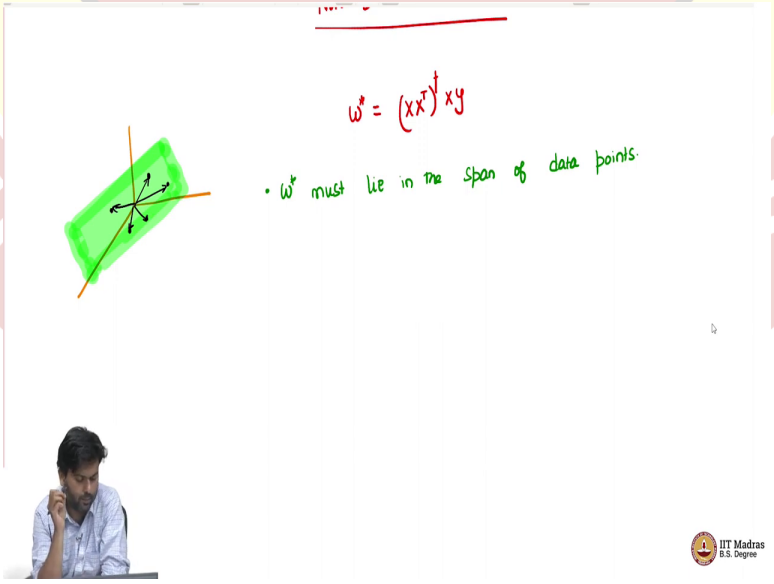
the dot products between data points. And once we were able to do that, then we could use a kernel matrix to replace this dot product. And then it would mean as if we went to a higher dimension and did a PCA that was our kernel PCA algorithm.

So, we are going to do something like this here as well. So, it is as if you want to map your data points to a higher dimension and then learn a linear model in high dimension a regressor. But then you do not want to explicitly compute these high dimensional mappings.

So, how can we do that, can we do that? We can do that, if our regression solution somehow can be written in terms of the dot products between data points, but what is the regression solution itself the regression solution itself is $(xx^T)^{\dagger}xy$.

So, this is the matrix this is the vector that we have and as it stands, it is it does not depend on the dot product directly. So, xx^T is a d x d matrix a dot product matrix would be an n cross n matrix where you have pair wise value for every pair of data points, that is not there here. But, but now, we know that this w* from our geometry considerations also and also by thinking about this a bit that our w* the solution to this this problem of linear regression must lie in the span of the data points.

(Refer Slide Time: 03:13)

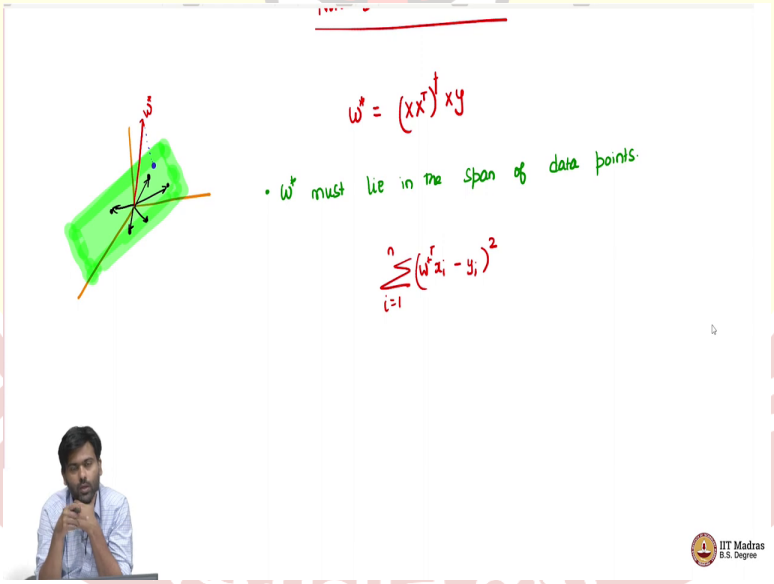


So, let me put that down and then try to explain that. So, w* this is the first point that we will note must lie in the span of data points. Now why is that true? So, what does it mean to say w* lies in the span of data points, so you have a bunch of data points d dimension, so then w* is also a vector in d dimension, I am saying w if all the data points were lying on a plane in 3 dimensional space, then w* will also be on the same plane.

So, so let us think of this in pictorially maybe you have a 3 (dime) 3 dimensional data points, but then necessarily not all points are in 3 dimension. Let us say there is some maybe use a different color maybe there is some 2 dimensional plane sitting inside this 3 dimensional space, which is where our w* actually lies in, sorry, that is where our data points lie in not w* at.

So, let us say all of our data points are they are 3 dimensional points, but then they effectively are lying in some 2 dimensional plane let us say. Now, now the question is, will our w* also lie on this plane? Or should can w* lie outside this plane?

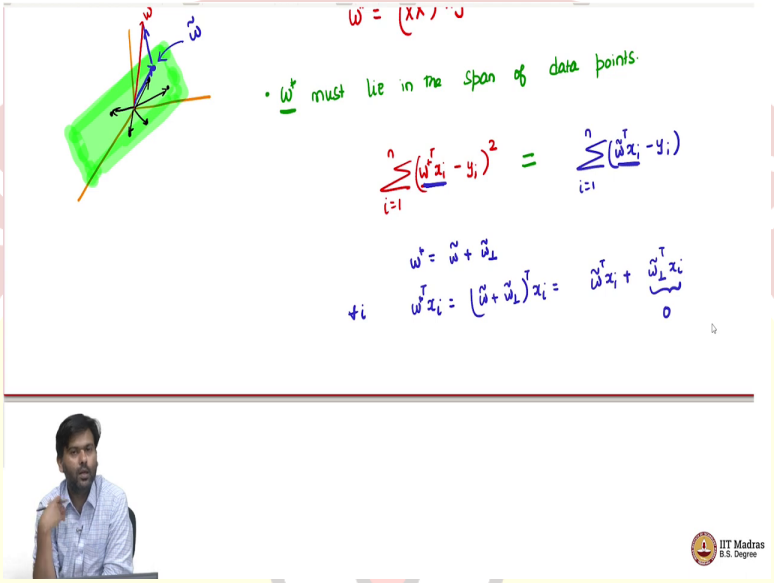
(Refer Slide Time: 04:50)



Now, let us say w* for the sake of argument, let us say the w* was lying outside this plane, maybe this was our w*. Now, remember, w* is the one that minimizes our minimizes the sum of squared error. So, $(w^{*T} xi - yi)^2$ is as small as possible among all possible w's.

Now, if this w* was actually out of this plane, so maybe it was not on this plane, then, let us now look at the following new w that I give you, which is the projection of this w* onto this plane that is the closest point of w* on this plane.

(Refer Slide Time: 05:35)



Now, let us call this guy as w~, now, let us ask the question what can we say about $\tilde{w}^T$ xi - yi, well, how does this compare to this guy. So, it is this of course, it cannot be less than this strictly less than this because by definition, we are assuming w* has the smallest value, because it solves this problem.

So, this is the right hand side cannot be less than this, but what can what is what is the actual relation between these two? Now, if you think about this, so, now w* projection onto this plane is w~. So, now, which means that there is this maybe I should do a different color. So, there is this this vector plus this vector that is w* can be written as w~ plus $\tilde{w}_\perp$. So, the one that is perpendicular to w~, and then if I add these two things, I get w*.

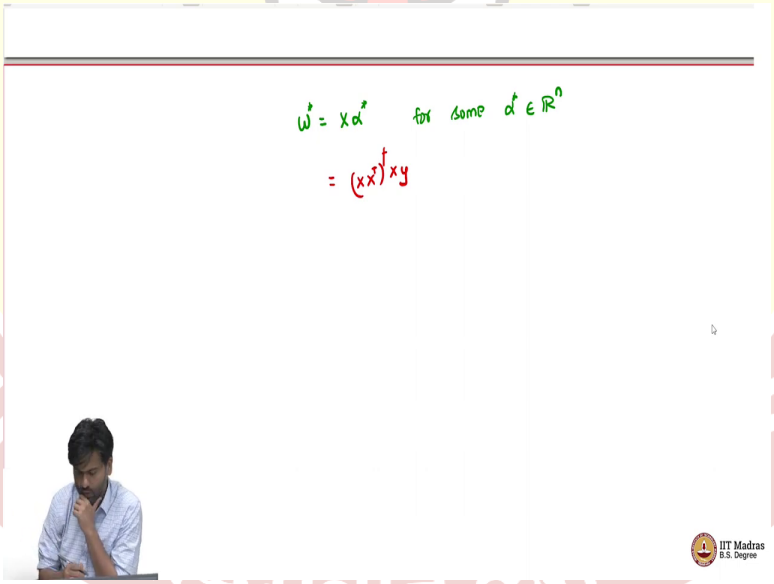
So, now $\tilde{w}_\perp$ is perpendicular to is perpendicular to w* to this plane basically, so, because it is you are projecting w onto this plane, so, which means w~ is perpendicular to this plane. So, it is perpendicular to all the points on this plane, it is orthogonal to all the points on this plane.

So, which means that $w^T xi$, $w^{*T} xi = w^{\sim} + w^{\sim T} xi$, which means, it is same as $w^{\sim T} xi + w^{\sim}_{\perp}{}^{T} xi$, but then because this is orthogonal to every in this plane xi in this plane, so, this guy is 0. So, this is for every i.

So, for any data point, the dot product that w* makes with respect to the data point is exactly same as the dot product that $w^{\sim}$ makes with respect to this data point. So, which means that w^{*T} xi is same as $w^{\sim T}$ x which means that the prediction of w* and $w^{\sim}$ for all the data points are exactly the same, which means that the error that both of these will suffer is exactly the same so, with respect to the training data which means, I might as well use $w^{\sim}$ as opposed to w*. So, as long as I have $w^{\sim}$ I am good enough.

So, what I can then say is that, well, without loss of generality, I can assume that the original w* that I have actually lies on the plane or on in general in the span of the data points itself. So, I can start with such a w* because that w* is good enough for my predictions and so, I will use that w*

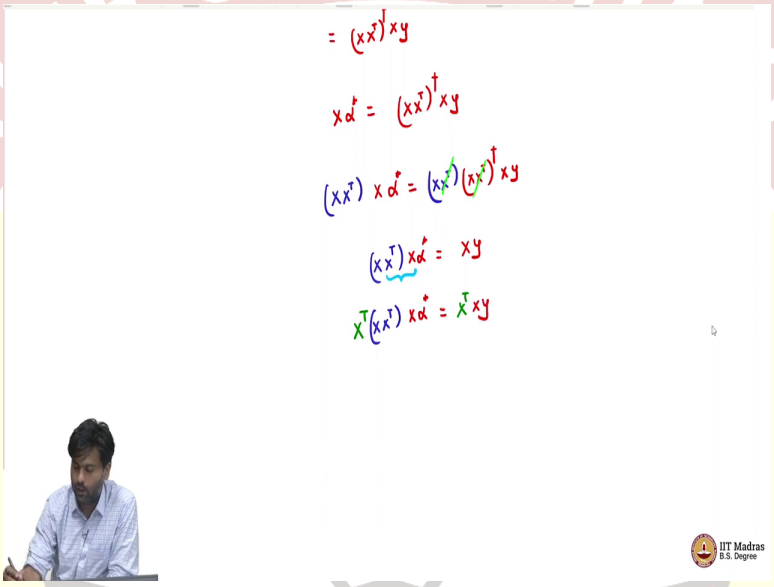
(Refer Slide Time: 08:30)



So, what does that mean? That means that w* equals some linear combination of your data points. So, your data points are in a plane and so, linear combination of these data points should have given w*.

So, it is some x times α* for some α* in Rn. So, Rn because there are n data points and we have a d x n matrix which is which is x. So, you are taking all the vectors in the dataset, now weighing them using alphas α* i for the ith the data point adding them up and then you will get w*. But remember we also know w* is just $(xx^T)^\dagger xy$ this comes from our gradient analysis. So, we took the gradient set it to 0 and then we got w w* is this.

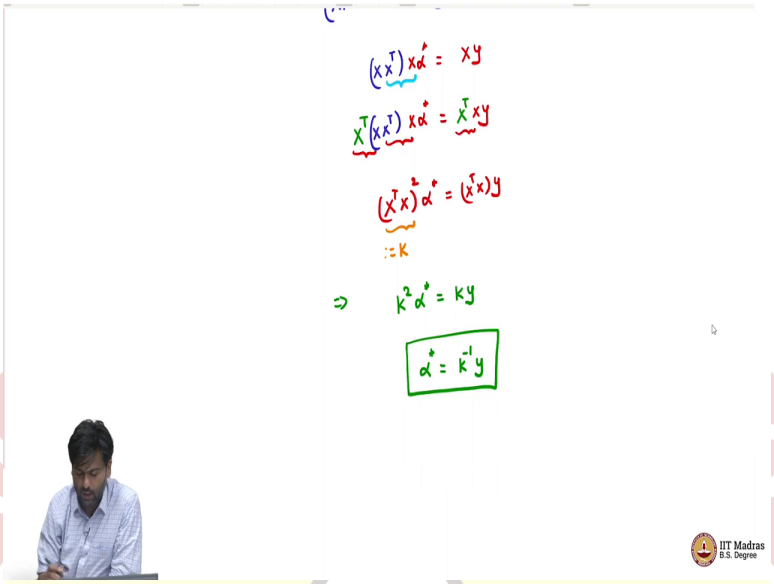
(Refer Slide Time: 09:25)

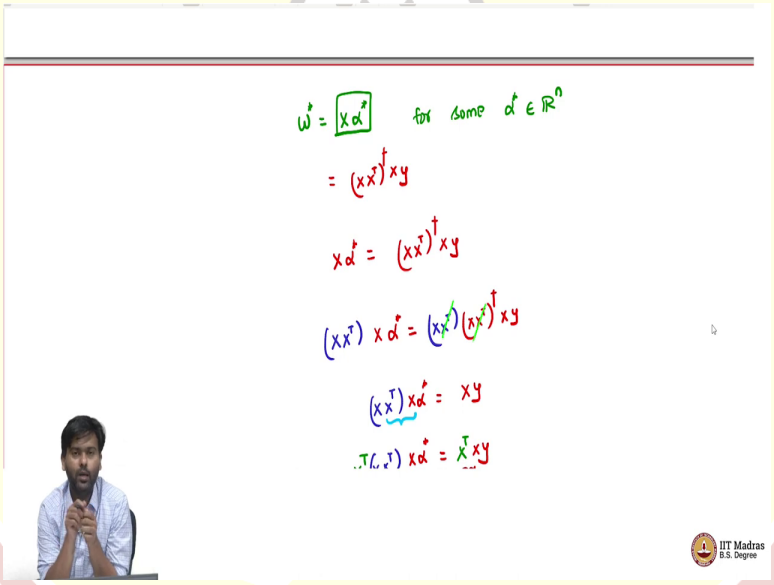


So, this means that x α star equals x x^T inverse x y or pseudo inverse, I mean I will keep saying inverse but then it also means pseudo inverse. Now, what we want to do is we will kind of try to get rid of this xx^T. So, we will pre multiply by xx^T on both sides to get $xx^Tx\alpha^* = (xx^T)(xx^T)^\dagger xy$. This is to cancel out this guy so these two guys kind of cancel out, because that is the property of pseudo inverse, you multiply the matrix it in pseudo inverse you will get an i.

So, this is, this just means we have $(xx^T) \times \alpha^* = xy$. This is where we are where we are right now. So, as you can see, the goal of the whole goal is to somehow write it in terms of x^Tx, so we have x^Tx here, but then there is also an x sitting on the other side, so we can simply say that, we will pre multiply this whole thing by x^T, let me use a different color to to show the difference maybe screen $x^T(x\, x^T)x\, \alpha^* = x^T\, x\, y$.

(Refer Slide Time: 11:02)



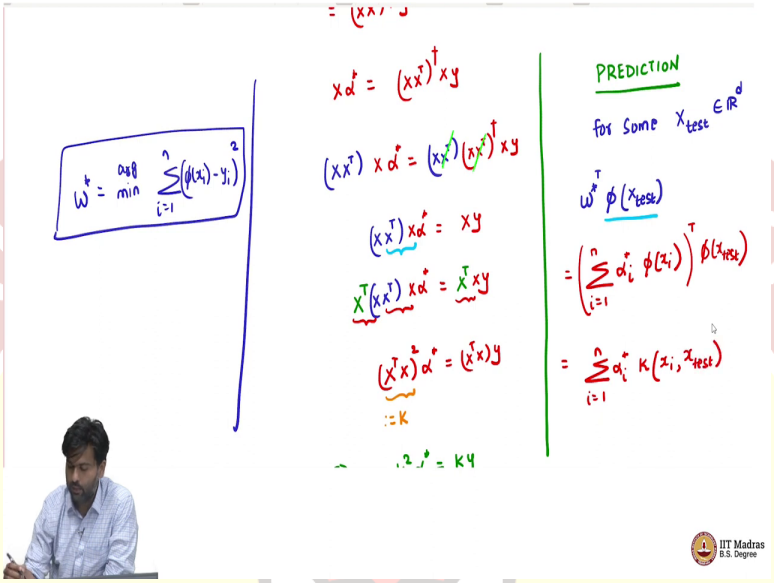


Now, we have something that is nice in the sense that we have x^Tx occurring twice here and occurring once here so, which means this whole thing can be written as $x\ (\ x^Tx)^2\ \alpha^* = x^Tx\ y$. Which means x^Tx is our kernel matrix, which we can let us call this as k define this as k so, then this just means we have equation $k^2\alpha^*$ equals k y. Or in other words, if k is invertible, so then we can simply say α^* is just k inverse y.

So, basically, this simply means that what is the what is the use of this, so, what are we saying we are solved for α^*, where w^* is how you should combine your data points using α^* to get w^*.

And we are saying if you use a gentle kernel matrix, then the way to combine is given by the inverse of the kernel matrix times y.

(Refer Slide Time: 12:09)



Now, what is the use of this? The whole point of doing this is that now when you do prediction, so if you if you have a new data point, let us say, for some x_{test}, if you have a new data point x_{test} in d dimension, then the kernel would tell you that, well, I need $w^{*T}\phi$ of x_{test}. So, I have to map my x test to high dimension and then take a dot product with w*, which is the best w* which minimizes the squared error of all the data points map to the higher dimensional space.

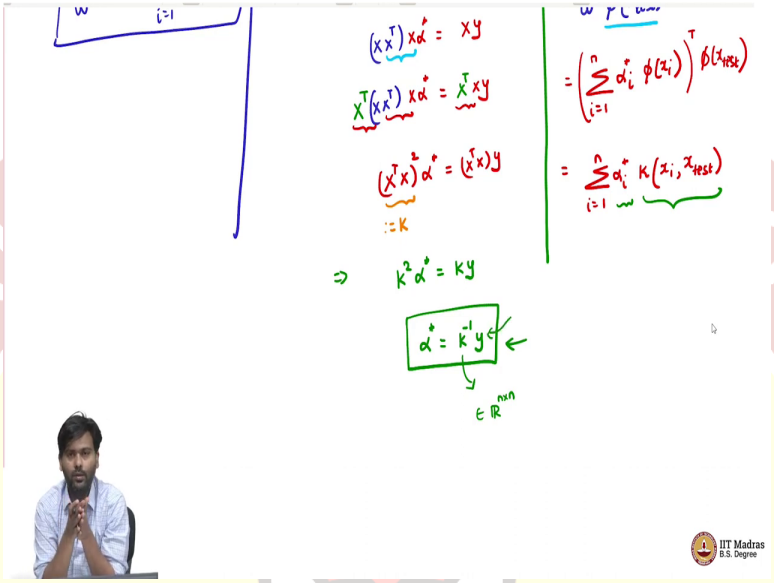
So, w* was would actually be the minimizer or the arg min $\Sigma^{n}_{i=1}((\phi xi)- yi)^2$. So, this w* is like you are given a data set, you are mapped every data point to high dimension, and then you are solving a linear regression problem in the high dimension, you get w*.

So, now, which means you if you get a new data point x_{test}, you map the data point to high dimension and then dotted with w*. However, we do not want to do this. So, we meaning we cannot afford to do this explicit mapping that is the theory of kernels, as we have seen before.

So, can we do make this prediction without explicitly using ϕ? Well, that is what our alphas will help us to. So, this is by the virtue of the fact that $w^* = \Sigma^{n}_{i=1}(\phi xi)$. So, that is what it means to say

w* is a linear combination of data points in high dimension. Now, this transpose ϕ of x_{test}, this is our prediction. But now this can be gotten as $\Sigma^{n}_{i=1}\alpha i^{*} K(xi, x_{test})$.

(Refer Slide Time: 14:28)



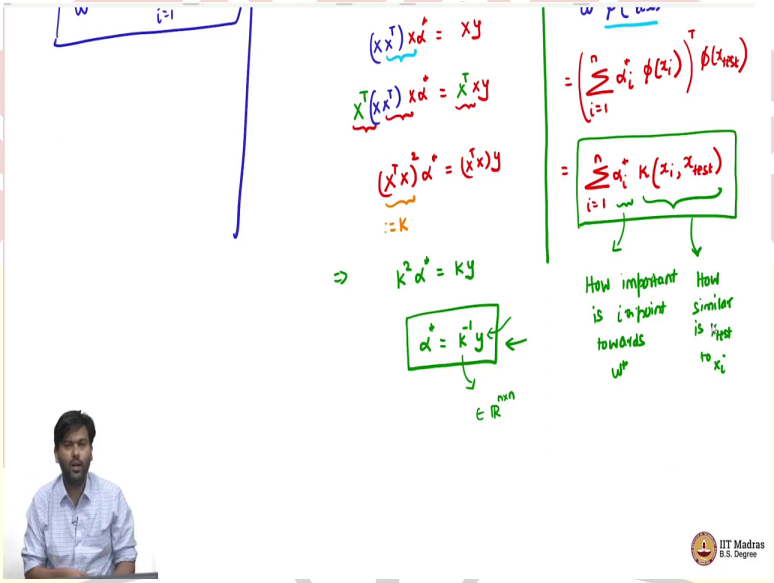
So, you have a kernel matrix, kernel function, which given any two data points evaluates the dot products in high dimension, which we can do using this and then α* is simply got by solving k inverse y, where k itself is just n cross n matrix, where you evaluate the kernel for the dataset, like how we did for PCA also, the just the extra thing that appears here is the y also matters now.

Of course, it should matter because it is not PCA, unsupervised so this is supervised. So, the label should also matter and so the y also plays comes into the picture here. So, basically, then how would you do linear and non-linear regression, the algorithm is very simple. So, you are given a bunch of data points, you are given a kernel function, now evaluate the kernel functions and all these data points to get the matrix k and n cross n matrix.

Now, compute $\alpha^{*} = k^{-1}y$. Now, once you have α* for a new new data point, any data point x_{test}, you you simply compute prediction as $\Sigma^{n}_{i=1}\alpha i^{*} K(xi, x_{test})$. it is as if asking you can think of the kernel function as how similar are these two data points, that is what the kernel function is, in essence doing.

So, in this it is saying, given a new test point, I am seeing how similar is this test point to each of my training data points, that is what the k(xi, x_{test}) is saying how (sim) how similar is this new point each of the training data points, but then how important is this training point to the w* that is given by α*.

(Refer Slide Time: 16:05)



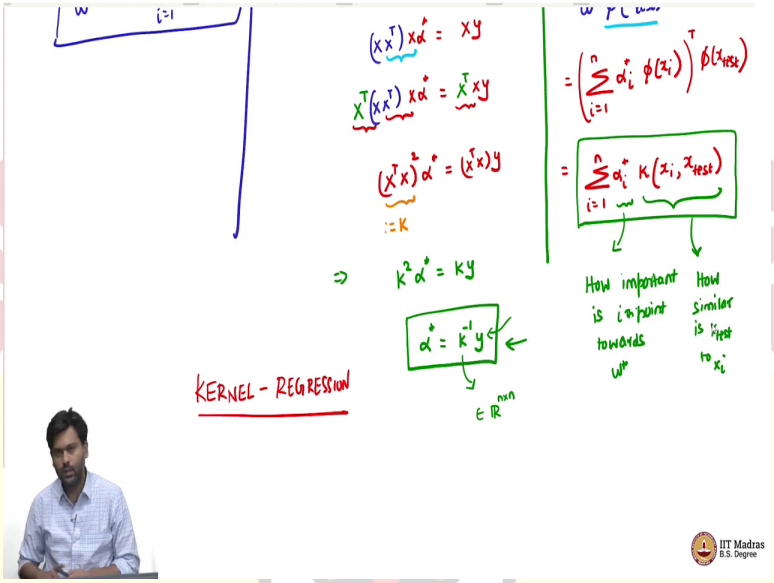
So, α* says how important is i ith point towards w*. Now, the k(xi, x_{test}) is says how similar is x_{test} to xi. So, you might be very similar to a data point which may not be so important for w*. So, it is the product of these two things that matters in our prediction, and then you sum it up over all data points.

So, now, by choosing different kernels, which we have seen earlier quadratic kernels, cubic kernels, Gaussian kernels, your favorite kernel, whatever you want to pick, now, you can solve the regression problem in any higher dimensional space. So, that is the power of kernels. And now, basically, what we have managed to do is we have kind of kernelized our linear regression problem to solve for non-linear regression as well.

So, this is the this is this is about the relationships between features and how they explain the variables. So, so, so, far we have looked at three different aspects. So, one is the geometry consideration of what does it mean to say we have a w* and we explain that as projection of

labels onto the features subspace spanned by the features is just your xw*, then we looked at computational considerations, we came up with a stochastic gradient descent algorithm.

(Refer Slide Time: 17:56)



And now, we are saying well, if you want to run if you want to capture non-linear relationships among features that explains the labels, then use kernel regression, this this is what is called as kernel regression as how PCA to kernel PCA regression to kernel regression.

Okay so, that is, so, these are some aspects. So, we will look at one more aspect now, about linear regression, which is another powerful aspect, which will actually lead us to a several more interesting variants of this, of this of this linear regression problem itself and that is to introduce something that we have not so far seen in linear regression, which is introducing probability into linear regression. Right now, everything is deterministic. So, what happens when you look at linear regression from a probabilistic viewpoint, which is what we'll see next.